Capstone Project Report

Third Semester

Submitted by

Bhaskar Ekansh P, 241EE213
Aarav Balaji, 241EE201
Chetan S K, 241EE214

Department of Electrical and Electronics Engineering
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
(NITK) SURATHKAL, MANGALORE - 575 025
November 27, 2025

Contents

INTRODUCTION

This report details the work carried out by our team as part of the preliminary capstone project preparation in Embedded Systems, Power Electronics, and Control Systems. The aim of this semester's tasks was to build a strong foundation in key areas before proceeding to more complex tasks for the final project.

The first task focused on understanding sensors and designing basic signal-conditioning circuits using operational amplifiers. The second task involved developing a linear regulated power supply using a 50 Hz transformer, diode rectifier, filter, and regulator. The third task explored numerical integration by simulating a harmonic oscillator using MATLAB or Octave. The fourth task introduced essential digital signal processing concepts by implementing simple DSP algorithms such as a low-pass filter.

Due to a demanding academic schedule this semester, we were unable to explore the full depth and extended scope of some tasks. However, we made sure to thoroughly understand the fundamentals and complete the core objectives of all four tasks. Broader exploration and deeper understanding of these concepts will be taken up in future semesters to complement this initial work.

The work presented in this report reflects the key concepts we learned, the practical skills we developed, and the foundational understanding we gained for future embedded-system and control-system projects.

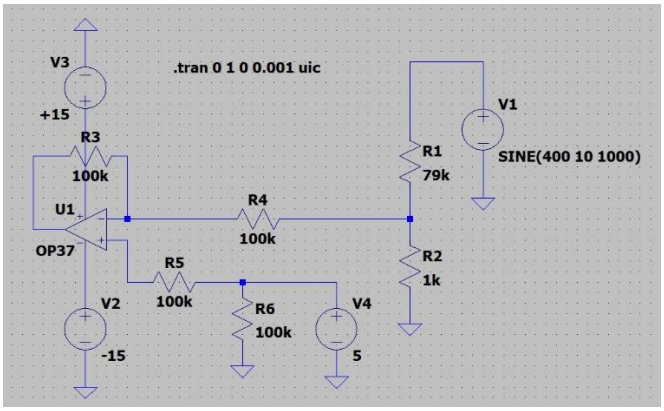
Task 1: Study of Sensors and Signal Conditioning

Overview

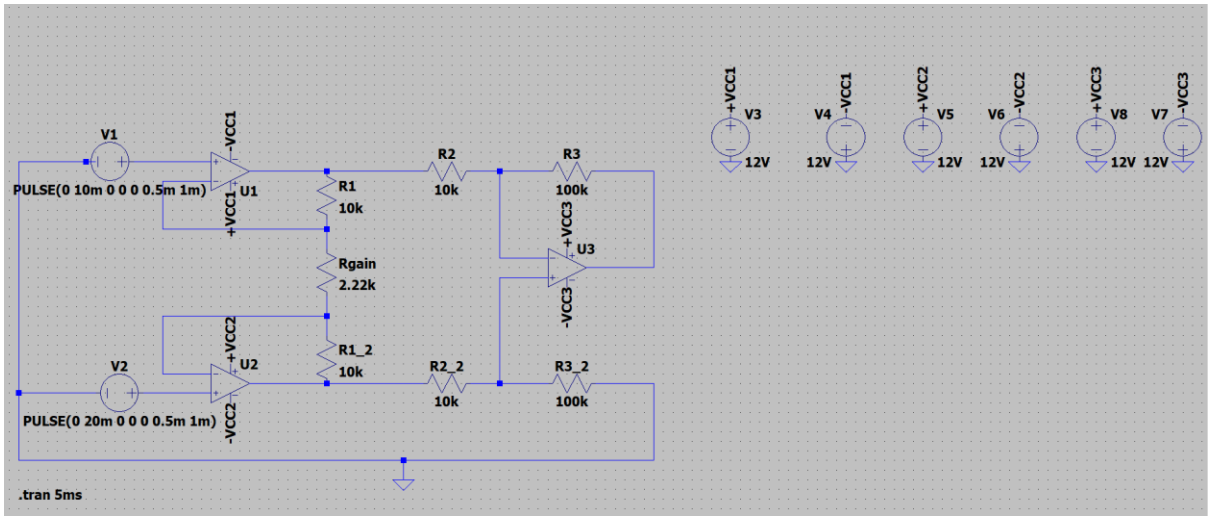
Voltage and current sensors are essential for measuring real-world quantities and interfacing them with analog and digital systems. Signal conditioning improves the usability of sensor outputs by scaling, filtering, and isolating signals. This task introduced the basic building blocks of analog signal conditioning, which form the foundation for reliable measurement systems.

Circuits Studied

1. Differential amplifier
    - A differential amplifier amplifies the difference between two input voltages while rejecting common-mode noise. This is essential when measuring signals across shunt resistors or in noisy environments.
    - Circuit schematic:



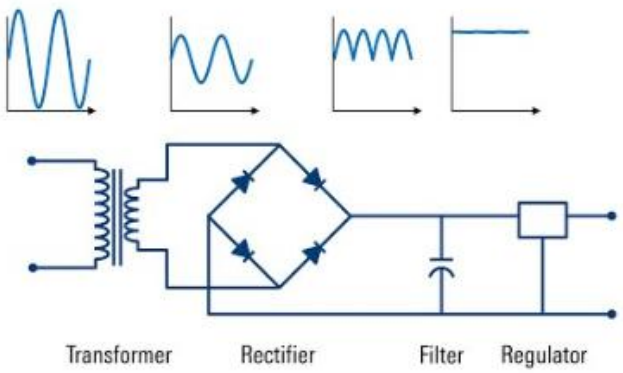
2. Instrumentation amplifier
    - Instrumentation amplifiers are widely used for accurate, low-noise amplification of small differential signals. They offer high common-mode rejection ratio and input impedance, making them suitable for sensor interfacing.
    - Circuit schematic:



Task 2: Linear Regulated Power Supply

Block Diagram

The regulated power supply consists of a transformer, diode bridge rectifier, smoothing filter, and linear regulator. It converts 50 Hz AC mains voltage to a stable DC output.



Circuit Explanation

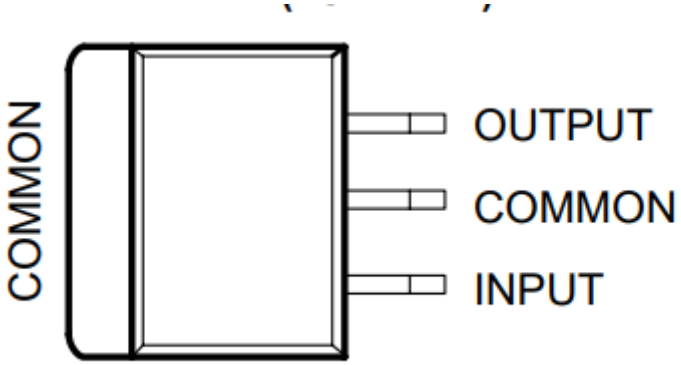
The circuit begins with a 230 V AC to 12 V AC step-down transformer that safely reduces the mains voltage to a level suitable for rectification.

The AC output is then fed into a full-wave bridge rectifier, which converts it into pulsating DC with a ripple frequency of 100 Hz.

To smooth this ripple, a large electrolytic capacitor, typically between 470 microF and 2200 microF, is connected across the output, storing charge and providing a more stable DC level.

This filtered voltage is then applied to a 7805 linear voltage regulator, which delivers a constant 5 V output as long as the input remains above its dropout voltage (around 7 V).

In practice, the capacitor value determines the amount of ripple, while the regulator ensures a stable, low-noise supply suitable for powering sensitive digital and analog circuits.



The 7805 is a three-terminal linear voltage regulator that provides a fixed +5 V DC output. Its pin configuration is: Pin 1 is the input, where an unregulated DC voltage is applied; Pin 2 is ground, which serves as the reference point for both input and output; and Pin 3 is the regulated 5 V output. Internally, the 7805 contains a series pass transistor, an error amplifier, a voltage reference, and built-in protection circuits.

Task 3: Numerical Integration Techniques

Understanding numerical integration techniques through application to a harmonic oscillator

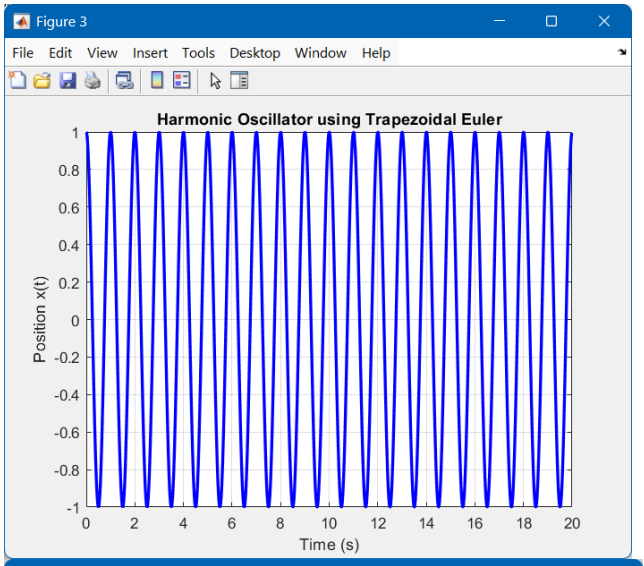
A harmonic oscillator is governed by the differential equation $d^2x/dt^2 + \omega^2x = 0$. Numerical integration methods such as Euler and Runge-Kutta allow simulation of its motion in MATLAB or Octave.

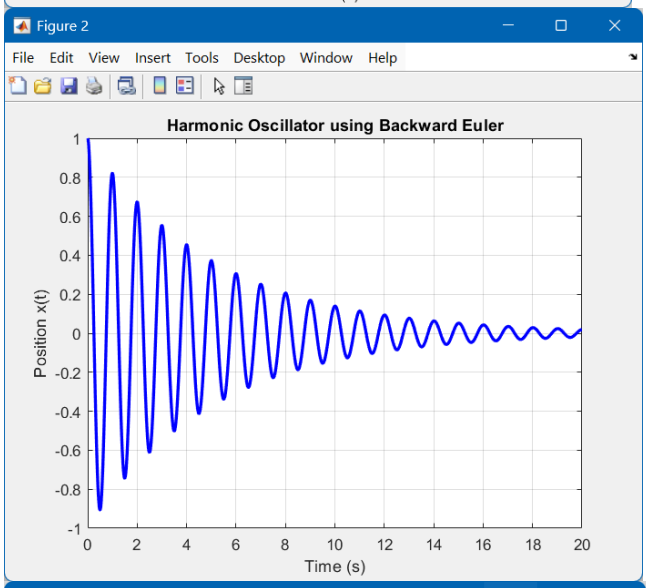
The three methods studied and simulated in this task include: the forward Euler, the backward Euler and the trapezoidal Euler.

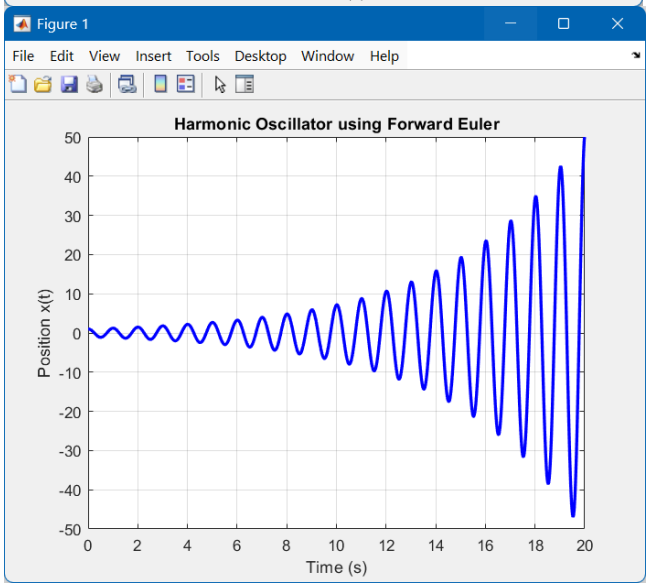
MATLAB Code for simulation of harmonic oscillator

```
clc; clear;
%% Parameters
omega0 = 2*pi*1;
dt = 0.01;
T = 20;
t = 0:dt:T-dt;
L = length(t);
x = zeros(1,L);
v = zeros(1,L);
%% Initial conditions
x(1) = 1;
v(1) = 0;
%% Forward Euler Integration
for i = 1:L-1
   x(i+1) = x(i) + dt * v(i);
   v(i+1) = v(i) - dt * (omega0^2 * x(i));
end
%% Plot
figure; plot(t, x, 'b', 'LineWidth', 2); xlabel('Time (s)'); ylabel('Position x(t)'); title('Harmonic Oscillator using Forward Euler'); grid on;
%% Backward Euler Integration
for i = 1:L-1
   x(i+1) = (x(i) + dt*v(i))/(1 + dt^2*omega0^2);
   v(i+1) = v(i) - dt * omega0^2 * x(i+1);
end
%% Plot
figure; plot(t, x, 'b', 'LineWidth', 2); xlabel('Time (s)'); ylabel('Position x(t)'); title('Harmonic Oscillator using Backward Euler'); grid on;
%% Trapezoidal Euler Integration
for i = 1:L-1
   x(i+1) = ((1 - (dt^2 * omega0^2)/4) * x(i) + dt * v(i)) / (1 + (dt^2 * omega0^2)/4);
   v(i+1) = v(i) - (dt/2) * omega0^2 * (x(i) + x(i+1));
end
%% Plot
figure; plot(t, x, 'b', 'LineWidth', 2); xlabel('Time (s)'); ylabel('Position x(t)'); title('Harmonic Oscillator using Trapezoidal Euler'); grid on;
```

Simulation Results







Observations

The Trapezoidal Euler method is a midpoint technique that averages the system's behaviour over each time step. It offers better accuracy and preserves energy much more effectively compared to the other two methods.

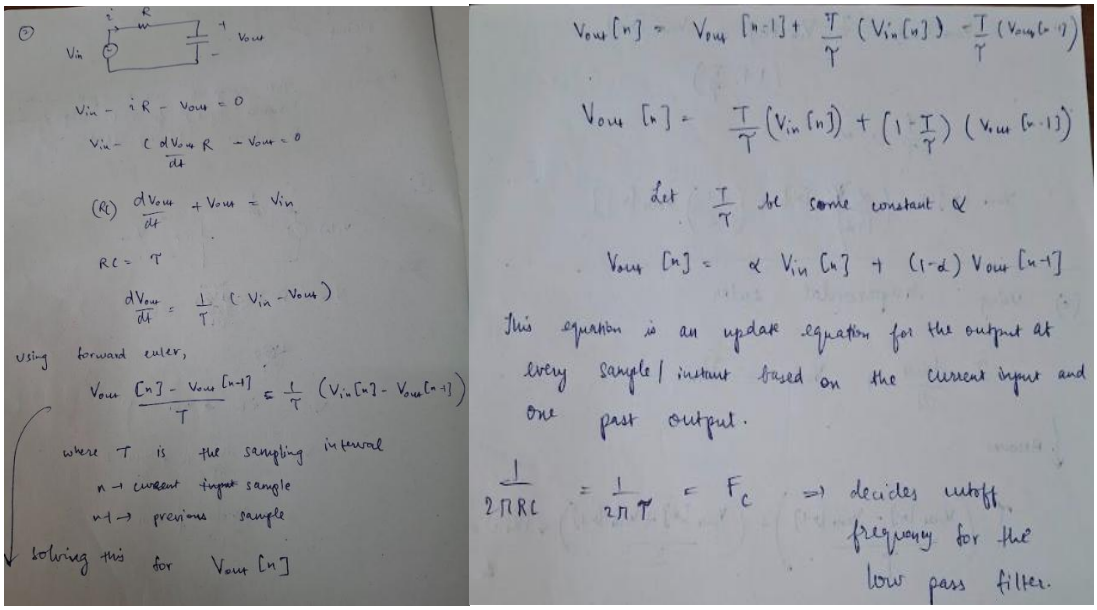
The Backward Euler method uses the future value of the state variables, resulting in an unconditionally stable scheme. However, for oscillatory systems, it introduces numerical damping. This causes the simulated motion to lose energy and the amplitude to decrease gradually

The Forward Euler scheme updates the position and velocity using the current values of the variables. It is simple but only conditionally stable. It tends to diverge for oscillatory systems due to energy gain.

Task 4: Emulation of DSP Algorithms

Digital filters and RMS/average algorithms are common DSP operations. MATLAB provides a convenient platform to demonstrate these algorithms. These can then be implemented on a microcontroller for processing of an actual signal.
For an example, we studied the implementation of a RC low pass filter, whose effect can be emulated in discrete time by discretizing the differential equation governing the system, and using the update equations so found to obtain a filtered output of a discrete signal.
First, we studied the relevant mathematics. Then we implemented the low-pass filter in MATLAB, and finally on the STM32G474RE, with the help of the STM32CubeIDE.

Overview of the discretization of the RC low-pass analog filter



The same procedure can be repeated using the Backward Euler and Trapezoidal Euler techniques to achieve a low-pass filtering algorithm

MATLAB implementation of the digital low-pass filter

```
clc;clear;
Fc = 5; % Cutoff frequency
Fs= 1000; % Sampling frequency
T = 1/Fs; % Sampling interval
%% Input Signal and initialization
t = 0:T:1 - T;
tau = 1/(2*pi*Fc);
alpha =  T / (T + tau);
a = (2*tau - T)/(2*tau + T);
b = T/(2*tau + T);
sig = @(t) 10 + 5*sin(50*t) + 5*sin(500*t);
x = sig(t);
L = length(x);
y_prev = 0 ;
```

```
y = zeros(size(x));
%% Filter implementation
% using forward euler
 for i = 1:L
          x_new = x(i);
          y_new= alpha*x_new + (1-alpha)*y_prev;
          y_prev = y_new;

    y(i) = y_new;
end

% using backward euler
for i = 1:L
    x_new = x(i);
    y_new= (alpha/(1 + alpha))*x_new + (1/(1+alpha))*y_prev;
    y_prev = y_new;
    y(i) = y_new;
 end

% using trapezoidal euler
x_prev = 0;
for i = 1:L
   x_new = x(i);
   y_new = a*y_prev + b*(x_new + x_prev);

   y(i) = y_new;
   y_prev = y_new;
   x_prev = x_new;
end

%% Plots
figure; subplot(2,1,1); plot(t,x); title('Original Signal'); xlabel('Time (s)'); ylabel('Amplitude');
subplot(2,1,2); plot(t,y); title('Filtered Signal'); xlabel('Time (s)'); ylabel('Amplitude');
```

Implementation on an embedded platform (the Nucleo-G474RE development board)

After validating the algorithm in MATLAB, we ported the filter to the STM32G474RE development board using STM32CubeIDE. The microcontroller sampled the input signal through the ADC at a fixed sampling rate, and the filtering equation was executed inside the main loop.
The same constants used in MATLAB were applied in the firmware.
The code used a variable to store the previous output value, mirroring the discrete-time update rule.
The final filtered output could be observed through DAC output.
The full implementation is available at:
https://github.com/Ekansh345/Implementation-of-dsp-algorithms-on-a-microcontroller

Future Work

Thess preliminary tasks focused on building foundational skills in sensors, power electronics, numerical simulation, and basic DSP implementation. While the core objectives for this semester were completed, several meaningful extensions can be pursued in future semesters to deepen our understanding and move toward a more advanced final capstone project.

1. Hardware implementation of higher-order filters:
Extending the first-order low-pass filter to second-order and higher-order designs, both in MATLAB and on the microcontroller, would help explore practical DSP filter characteristics.

2. Real-time data acquisition using ADC + DMA:
Implementing DMA-based sampling instead of the current polling-based sampling on the STM32 would enable better understanding of real-time DSP tasks.

3. Complete hardware prototype of the linear power supply:
Building a fully tested PCB version of the regulated supply, including protection circuits and load-testing, would help in learning PCB design.

4. Exploration of more advanced numerical solvers:
Studying higher-order Runge–Kutta methods and comparing them with Euler-based techniques would improve simulation accuracy and prepare us for more complex modeling tasks.

Together, these extensions and other tasks assigned in the next semester will help transition the foundational learning of this semester into a more advanced capstone project in the coming year.